

Assert.js v1.2.2 cheatsheet - <https://github.com/Serrin/assert.js>

Category	Details	
Import assert	import assert from "../assert.js"; globalThis.assert = assert;	
Import assert as default	import { default as assert } from "../assert.js"; globalThis.assert = assert;	
Import assert as defaultExport	import defaultExport from "../assert.js"; globalThis.assert = defaultExport;	
Dynamic import	const assert = await import("../assert.js"); globalThis.assert = defaultExport;	
Config	assert.config.alwaysStrict; Default value is false. If value is true, then `assert.equal();` will be replaced with `assert.strictEqual();` and `assert.notEqual();` will be replaced with the `assert.notStrictEqual();`.	
Testrunner functions	type TResult<T> = {ok: true, value: T, block: Function, name: string} {ok: false, error: Error, block: Function, name: string};	
	assert.testSync(name = "assert.testSync", block): TResult	
	assert.test(name = "assert.testSync", block): TResult	
	assert.it(name = "assert.testSync", block): TResult	
	await assert.testAsync(name = "assert.testAsync", block): TResult	
class assert.TestSuite	assert.testCheck(result: TResult): result.ok is true	
	Add(...args: Array<TResult>): this;	success(): IterableIterator<TResult>;
	clear(): this;	failed(): IterableIterator<TResult>;
	get size(): number;	values(): IterableIterator<TResult>;
	toArray(): Array<TResult>	[Symbol.iterator]()
Argument types	type message = string Error;	
	type ExpectedType = string function Array<string function>;	
Constants	assert.VERSION;	
Errors (class)	assert.AssertionError([options: {[message], [cause], [actual], [expected], [operator]]});	
Exceptions	assert.throws(fn, [ErrorType string RegExp][,message]): Promise;	
	await assert.rejects(asyncFnOrPromise, [ErrorType string RegExp][,message]);	
	await assert.doesNotReject(asyncFnOrPromise, [ErrorType string RegExp][,message]);	
Basic	assert(condition[,message]);	assert.fail([message]);
		assert.fail([actual,expected,message:string,operator]);
	assert.ok(condition[,message]);	assert.notOk(condition[,message]);
Equality	assert.equal(actual,expected[,message]);	assert.notEqual(actual,expected[,message]);
	assert.strictEqual(actual,expected[,message]);	assert.notStrictEqual(actual,expected[,message]);
	assert.deepEqual(actual,expected[,message]);	assert.notDeepEqual(actual,expected[,message]);
	assert.deepStrictEqual(actual,expected[,message]);	assert.notDeepStrictEqual(actual,expected[,message]);
	assert.oneOf(value,collection[,message]);	assert.notOneOf(value,collection[,message]);

Comparasion	<code>assert.lt (value1, value2 [,message]);</code>	<code>assert.lte (value1, value2 [,message]);</code>
	<code>assert.gt (value1, value2 [,message]);</code>	<code>assert.gte (value1, value2 [,message]);</code>
	<code>assert.inRange (value, min, max [,message]);</code>	<code>assert.notInRange (value, min, max [,message]);</code>
	<code>assert.operator (value1, operator, value2 [,message]);</code>	Operator values: "=", "!", "==", "!=", "<", "<=", ">", ">=", "Object.is", "!Object.is"
String	<code>assert.match (string, regexp [,message]);</code>	<code>assert.doesNotMatch (string, regexp [,message]);</code>
	<code>assert.stringContains (actual, substring [,message]);</code>	<code>assert.stringNotContains (actual, substring [,message]);</code>
	<code>assert.stringStartsWith (actual, substring [,message]);</code>	<code>assert.stringNotStartsWith (actual, substring [,message]);</code>
	<code>assert.stringEndsWith (actual, substring [,message]);</code>	<code>assert.stringNotEndsWith (actual, substring [,message]);</code>
Object	<code>assert.includes (container, {keyOrValue, [value]} [,message]);</code>	<code>assert.doesNotInclude (container, {keyOrValue, [value]} [,message]);</code>
	<code>assert.isEmpty (value [,message]);</code>	<code>assert.isNotEmpty (value [,message]);</code>
Type	<code>assert.is (value, expectedType: ExpectedType [,message]);</code>	<code>assert.isNot (value, expectedType: ExpectedType [,message]);</code>
	<code>assert.typeOf (value, expectedType: string [,message]);</code>	<code>assert.notTypeOf (value, expectedType: string [,message]);</code>
	<code>assert.instanceOf (value, expectedType: function [,message]);</code>	<code>assert.notInstanceOf (value, expectedType: function [,message]);</code>
	<code>assert.isPrimitive (value [,message]);</code>	<code>assert.isNotPrimitive (value [,message]);</code>
	<code>assert.isNullish (value [,message]);</code> <code>assert.ifError (value [,message]);</code>	<code>assert.isNotNullable (value [,message]);</code>
	<code>assert.isNull (value [,message]);</code>	<code>assert.isNotNull (value [,message]);</code>
	<code>assert.isUndefined (value [,message]);</code>	<code>assert.isDefined (value [,message]);</code>
	<code>assert.isNumber (value [,message]);</code>	<code>assert.isNotNumber (value [,message]);</code>
	<code>assert.isNaN (value [,message]);</code>	<code>assert.isNotNaN (value [,message]);</code>
	<code>assert.isInt (value [,message]);</code>	<code>assert.isNotInt (value [,message]);</code>
	<code>assert.isFloat (value [,message]);</code>	<code>assert.isNotFloat (value [,message]);</code>
	<code>assert.isString (value [,message]);</code>	<code>assert.isNotString (value [,message]);</code>
	<code>assert.isBigInt (value [,message]);</code>	<code>assert.isNotBigInt (value [,message]);</code>
	<code>assert.isBoolean (value [,message]);</code>	<code>assert.isNotBoolean (value [,message]);</code>
	<code>assert.isTrue (value [,message]);</code>	<code>assert.isNotTrue (value [,message]);</code>
	<code>assert.isFalse (value [,message]);</code>	<code>assert.isNotFalse (value [,message]);</code>
	<code>assert.isSymbol (value [,message]);</code>	<code>assert.isNotSymbol (value [,message]);</code>
	<code>assert.isFunction (value [,message]);</code>	<code>assert.isNotFunction (value [,message]);</code>
	<code>assert.isObject (value [,message]);</code>	<code>assert.isNotObject (value [,message]);</code>